Introduction of Object Oriented Programming

- As the name suggests, Object-Oriented Programming or OOPs refers to languages that use objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

OOPs Concepts:

- Class
- Objects
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

1. Class:

A class is a user-defined data type. It consists of data members and member functions, which can be accessed and used by creating an instance of that class. It represents the set of properties or methods that are common to all objects of one type. A class is like a blueprint for an object.

For Example: Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, mileage are their properties.

2. Object:

It is a basic unit of Object-Oriented Programming and represents the real-life entities. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated. An object has an identity, state, and behavior. Each object contains data and code to manipulate the data. Objects can interact without having to know details of each other's data or code, it is sufficient to know the type of message accepted and type of response returned by the objects.

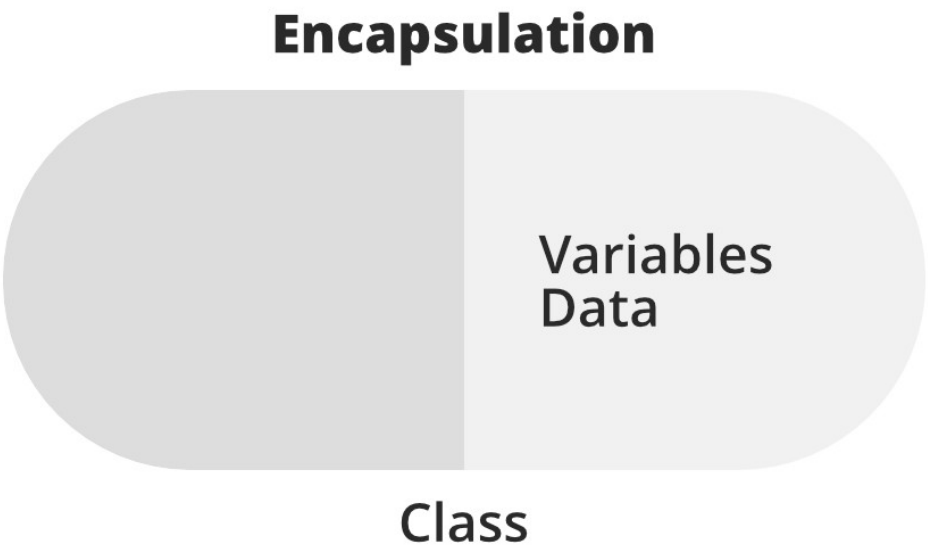
3. Data Abstraction:

Data abstraction is one of the most essential and important features of object-oriented programming. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. Consider a real-life example of a man driving a car. The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

4. Encapsulation:

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. In Encapsulation, the variables or data of a class are hidden from any other class and can be accessed only through any member function of their

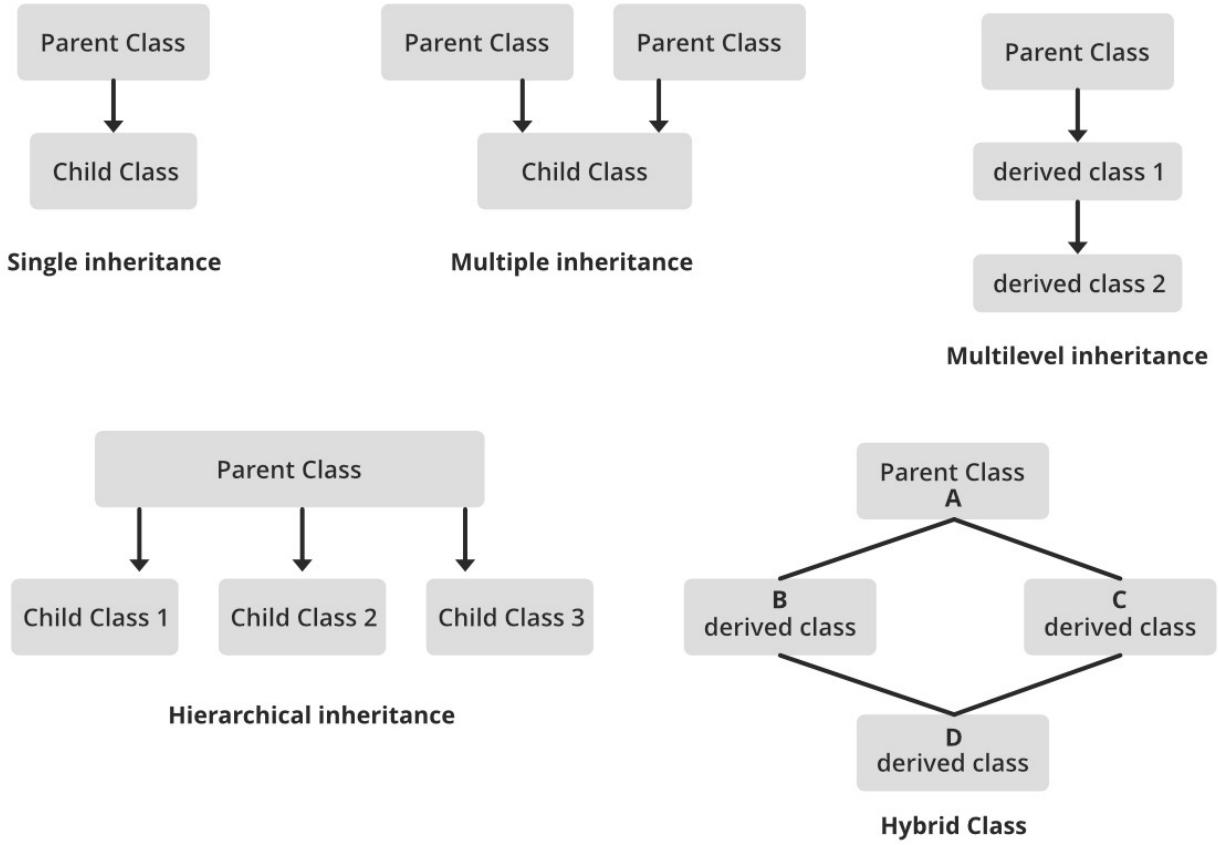
class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as **data-hiding**.



Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name “sales section”.

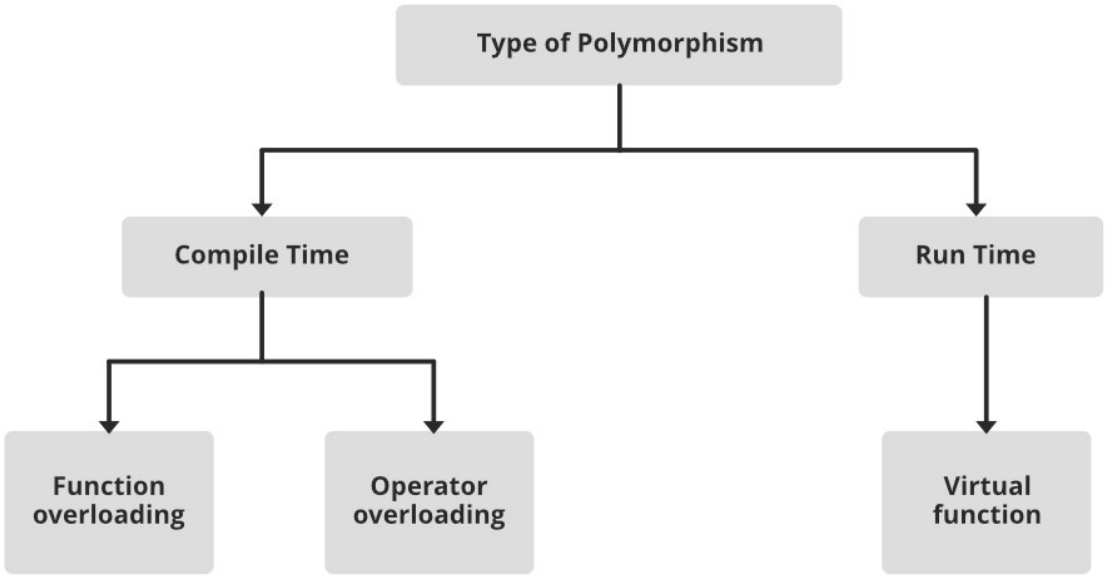
5. Inheritance:

Inheritance is an important pillar of OOP(Object-Oriented Programming). The capability of a class to derive properties and characteristics from another class is called Inheritance. When we write a class, we inherit properties from other classes. So when we create a class, we do not need to write all the properties and functions again and again, as these can be inherited from another class that possesses it. Inheritance allows the user to reuse the code whenever possible and reduce its redundancy.



6. Polymorphism:

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. For example, A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person posses different behavior in different situations. This is called polymorphism.



7. Dynamic Binding:
In dynamic binding, the code to be executed in response to the function call is decided at runtime. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time. Dynamic Method Binding One of the main advantages of inheritance is that some derived class D has all the members of its base class B. Once D is not hiding any of the public members of B, then an object of D can represent B in any context where a B could be used. This feature is known as subtype polymorphism.

8. Message Passing:
It is a form of communication used in object-oriented programming as well as parallel programming. Objects communicate with one another by sending and receiving information to each other. A message for an object is a request for execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results. Message passing involves specifying the name of the object, the name of the function, and the information to be sent.

Basic Components of a C++ Program and Program Structure

Introduction

A C++ program consists of various components that work together to execute a task. Understanding these components and the structure of a C++ program is essential for

Basic Components of a C++ Program

A C++ program consists of the following essential components:

1. Preprocessor Directives (#include)

- These are instructions that tell the compiler to include necessary files before compilation.
- The most commonly used directive is #include <iostream>, which includes the **input-output stream** for handling standard input and output.
  Example:

```
#include <iostream> // Allows input-output operations
```

---

2. Namespace (using namespace std;)

- The std namespace contains standard C++ library functions.
- Using using namespace std; allows us to use standard functions like cout and cin without prefixing them with std::.
  Example:

```
using namespace std;
```

Alternatively, instead of using namespace std;, we can use std::cout and std::cin explicitly.

---

3. Main Function (int main())

- The main() function is the entry point of any C++ program.
- The program starts execution from main() and terminates when return 0; is encountered.
  Example:

```
int main() {
return 0;
}
```

---

4. Input and Output Statements

- cout is used to print output to the console.
- cin is used to take input from the user.
  Example:

```
int x;
cout << "Enter a number: ";
cin >> x;
cout << "You entered: " << x;
```

---

5. Variables and Data Types

- Variables store data that can be used in a program.
- Data types define the type of data stored.
  Example:

```
int age = 25; // Integer variable
double pi = 3.14; // Floating-point variable
char grade = 'A'; // Character variable
```

---

6. Operators

- Operators perform arithmetic, comparison, and logical operations.
  Example:

```
int a = 5, b = 3;
int sum = a + b; // Addition
bool isGreater = (a > b); // Comparison
```

---

7. Control Statements (if-else, loops)

- if-else statements help in decision-making.
- Loops (for, while, do-while) allow repetitive execution.
  Example:

```
if (age > 18) {
cout << "You are an adult.";
} else {
cout << "You are a minor.";
}
```

---

8. Functions

- Functions allow code reuse and modularity.
- They can have parameters and return values.
  Example:

```
int add(int a, int b) {
return a + b;
}

int main() {
cout << add(5, 10); // Output: 15
}
```

---

9. Classes and Objects

- C++ supports Object-Oriented Programming (OOP) using **classes** and **objects**.
  Example:

```
class Car {
public:
string brand;
void showBrand() {
cout << "Car brand: " << brand;
}
};

int main() {
Car myCar;
myCar.brand = "Toyota";
myCar.showBrand();
}
```

C++ Program Structure

A basic C++ program follows this structure:

Programming

```
// 1. Preprocessor directives
#include <iostream>

// 2. Namespace
using namespace std;

// 3. Function declaration (optional)
int add(int, int);

// 4. Main function
int main() {
int a, b;

// 5. Input from user
cout << "Enter two numbers: ";
```

```
cin >> a >> b;

// 6. Function call
cout << "Sum: " << add(a, b) << endl;

return 0;
}

// 7. Function definition
int add(int x, int y) {
return x + y;
}
```

Compiling and Executing C++ Program.

Compiling is the process of translating human-readable **source code** into machine-readable **machine code** (an executable file), while **executing** is the act of the computer's processor running that machine code to perform its intended tasks.

Medium +1

Compilation vs. Execution in C++

Feature	Compilation	Execution
Primary Purpose	To translate source code (e.g., `.cpp` files) into an executable format.	To run the already translated machine instructions on the CPU to produce output.
Input	Source code files and necessary library definitions.	The executable file produced by the compilation and linking process.
Output	Object files and eventually a single, complete executable file (e.g., `.exe` on Windows, `a.out` on Linux).	The program's actual result, such as text on the screen, file manipulation, or user interaction.
Errors Detected	Detects **compile-time errors**, primarily syntax errors (e.g., missing semicolons, unmatched parentheses) and some semantic errors.	Detects **run-time errors** (e.g., division by zero, accessing an invalid memory address), which cause abnormal program termination.

Timing	Occurs once, before the program can be run.	Occurs after successful compilation, and can be repeated multiple times without recompiling (unless the source code is changed).

The C++ Program Flow

The full process involves several steps, where compilation is a major phase before execution can occur:

Filo +3

- **Preprocessing**: Handles directives like `#include` and `#define` to prepare the code for the compiler.
- **Compilation**: Translates the preprocessed code into assembly language, then into machine code, resulting in an object file.
- **Linking**: Combines the object file(s) with necessary library files to resolve references and create a single, complete executable file.
- **Execution**: The operating system loads the resulting executable file into memory and the CPU begins running the instructions to carry out the program's logic.

Procedural Programming vs Object-Oriented Programming

Below are some of the differences between procedural and object-oriented programming:

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
Procedural programming follows a ***top-down approach***.	Object-oriented programming follows a ***bottom-up approach***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.

Procedural Oriented Programming	Object-Oriented Programming
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.
In procedural programming, overloading is not possible.	Overloading is possible in object-oriented programming.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.
Procedural programming is used for designing medium-sized programs.	Object-oriented programming is used for designing large and complex programs.
Procedural programming uses the concept of procedure abstraction.	Object-oriented programming uses the concept of data abstraction.
Code reusability absent in procedural programming,	Code reusability present in object-oriented programming.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#, etc.